SOFTWARE ENGINEERING

LiviSync

Smart Roommate Matching Android App

Deliverable #2 — Iteration 1 / Sprint 1 Documentation

Group Members

Awais Akhtar | 23L-0792
Bissam Iftikhar | 23L-0913
Muaz Arshad | 23L-0816

Platform: Android (Java + XML)
IDE: Android Studio | Backend: Firebase (Auth + Firestore)

Date of Submission: 24-03-2026

1. Project Title & Introduction

Project Title: LiviSync – Smart Roommate Matching Platform

LiviSync is a native Android mobile application designed to help university students find compatible roommates based on their lifestyle habits and personal preferences. University students often struggle to find suitable roommates when transitioning into shared housing, hostels, or dormitories. Existing solutions typically only list available rooms without evaluating personal compatibility, lifestyle habits, or preferences such as sleeping schedules, cleanliness standards, budget ranges, and policies around guests, pets, or smoking.

LiviSync addresses this gap by allowing students to build detailed lifestyle profiles and using a smart compatibility scoring algorithm to suggest the most suitable matches. The platform also supports real-time messaging, match request management, push notifications, reporting, blocking, and an admin moderation panel.

Sprint 1 (Iteration 1) covers the foundational module of the application: project setup and configuration, Firebase Authentication, a splash screen, student profile creation, multi-step lifestyle preference setup, profile edit and view screens, Firestore data modelling, and bottom navigation bar scaffolding. The deliverable of this sprint is a working APK in which users can register, log in, and build their complete lifestyle profile.

2. Sprint Backlog — Sprint 1

2a. Module Name

Module 1: Foundation, Authentication & Profiles

2b. User Stories

The following user stories were selected and developed during Sprint 1. Each story follows the standard user story template with sub-stories and acceptance criteria.

US-01: User Registration

Story ID: US-01 Story Title: User Registration	**Priority:** High **Estimated Hours:** 4
User Story: As **a student,** I want **to register with my email and password** so that **I can create an account on LiviSync.** **Description:** This story covers new student account creation using Firebase Authentication with email and password. On successful registration the user is redirected to the profile setup flow. **Sub User Stories:** • As a student, I want to enter my email and a strong password to create an account. • As a student, I want to see inline validation messages when I enter invalid credentials. • As a student, I want to be automatically redirected to profile setup after registering. • As a student, I want my account to be securely stored via Firebase Authentication. **Acceptance Criteria:** • Tapping Register with a valid email and password creates a Firebase Auth account. • An error message is displayed if the email is already in use. • A password shorter than 6 characters triggers an inline validation error. • On success, the user is navigated to ProfileSetupActivity.	

US-02: User Login

Story ID: US-02 Story Title: User Login	**Priority:** High **Estimated Hours:** 3
User Story: As **a student,** I want **to log in with my email and password** so that **I can access my account and the app features.** **Description:** This story covers authentication of existing users. Firebase Auth validates credentials and the app maintains the session so the user does not need to log in on every launch. **Sub User Stories:** • As a student, I want to enter my email and password to log in. • As a student, I want to see an error message if my credentials are incorrect. • As a student, I want to be directed to the home screen after a successful login. • As a student, I want the app to remember my session so I do not log in every time. **Acceptance Criteria:** • Entering correct credentials navigates the user to MainActivity. • Incorrect credentials display a descriptive error toast. • The app detects an active session on launch and skips the login screen. • The Login and Sign Up forms are presented as tabs within a single ViewPager2 screen.	

US-03: Splash Screen

Story ID: US-03 Story Title: Splash Screen	**Priority:** Medium **Estimated Hours:** 2
User Story: As **a student,** I want **to see an animated splash screen on app launch** so that **the app feels polished and routes me to the correct screen.** **Description:** This story covers the animated launch screen that displays the LiviSync logo and slogan. It also checks the Firebase Auth	

session state and routes the user to either the login screen or the home screen accordingly. **Sub User Stories:** • As a user, I want to see the LiviSync logo and slogan during app launch. • As a user, I want a smooth fade-in animation on the splash screen. • As a returning user, I want the app to check my session and take me directly to home. **Acceptance Criteria:** • The splash screen displays for approximately 2 seconds before routing. • A fade animation plays on the title and slogan text. • Authenticated users are routed to MainActivity; unauthenticated users to LoginActivity. • The splash screen is not shown again once the user is on the home screen.	

US-04: Student Profile Creation

Story ID: US-04 Story Title: Student Profile Creation	**Priority:** High **Estimated Hours:** 6
User Story: As **a student,** I want **to create a personal profile with my name, age, bio, photo, and gender** so that **other students can learn about me.** **Description:** This story covers the first-time profile creation flow where a newly registered student fills in their personal information. The data is persisted to the Firestore users collection under the user's UID. **Sub User Stories:** • As a student, I want to enter my full name and age during profile setup. • As a student, I want to upload a profile photo from my device gallery. • As a student, I want to write a short bio to introduce myself to potential roommates. • As a student, I want to select my gender from a dropdown list. • As a student, I want my profile data saved to Firestore for future access.	

Acceptance Criteria: • Submitting the form with a valid name and age creates a document in the users Firestore collection. • The name field is mandatory; leaving it empty shows a validation error. • A default avatar is used if the user skips the photo upload. • After saving, the user is navigated to the lifestyle preferences setup screen.	

US-05: Multi-Step Lifestyle Preference Setup

Story ID: US-05 Story Title: Lifestyle Preference Setup	**Priority:** High **Estimated Hours:** 7
User Story: As **a student,** I want **to configure my lifestyle preferences through a guided setup** so that **the system can calculate accurate compatibility scores for me.** **Description:** This story covers the PreferencesActivity where users define their lifestyle attributes. These preferences are the primary input for the compatibility scoring engine built in Iteration 2. **Sub User Stories:** • As a student, I want to select my sleep schedule (Early Bird / Night Owl / Flexible). • As a student, I want to set my cleanliness rating on a scale of 1 to 5. • As a student, I want to specify my monthly budget range in PKR. • As a student, I want to set my preferred city or area. • As a student, I want to define my policies on smoking, pets, and guests. • As a student, I want my preferences persisted to the Firestore preferences collection. **Acceptance Criteria:** • All preference fields are saved atomically to the Firestore preferences collection on submission. • Empty budget fields trigger a validation error before saving. • The screen is pre-filled with existing values when accessed from Edit Profile.	

• On successful save, the user is navigated to the home screen.	

US-06: Profile Edit & View

Story ID: US-06 Story Title: Profile Edit & View	**Priority:** High **Estimated Hours:** 5
User Story: As **a student,** I want **to view and edit my profile information and preferences** so that **I can keep my details up to date.** **Description:** This story covers the ProfileFragment which displays the current user's data, and EditProfileActivity which allows the user to modify any profile or preference field. Changes are batch-written to Firestore. **Sub User Stories:** • As a student, I want to view my profile summary including bio, preferences, and photo. • As a student, I want to tap Edit Profile and update any field. • As a student, I want my changes saved in real time to Firestore. • As a student, I want the profile view to reload immediately after saving edits. **Acceptance Criteria:** • The Profile tab displays the authenticated user's name, bio, sleep schedule, cleanliness, budget, and policies. • Tapping Edit Profile opens EditProfileActivity pre-filled with current values. • Saving triggers a Firestore batch write to both the users and preferences collections. • On success, a toast confirms the update and the user is returned to the profile view.	

US-07: Firestore Data Models

Story ID: US-07 Story Title: Firestore Data Models	**Priority:** High **Estimated Hours:** 4
User Story:	

As **a student,** I want **well-defined Firestore data models for users and preferences** so that **all application data is consistently structured and queryable.** **Description:** This story covers the design and implementation of the Firestore schema including the users collection and preferences collection. Security rules are also configured to restrict access to authenticated users. **Sub User Stories:** • As a developer, I want a users collection with fields: uid, name, age, bio, gender, photoUrl, city. • As a developer, I want a preferences collection with: sleep, cleanliness, budgetMin, budgetMax, smoking, pets, guests. • As a developer, I want Firestore security rules restricting read/write to authenticated users only. **Acceptance Criteria:** • Both collections are created and populated correctly when a user completes profile setup. • Unauthenticated read or write attempts to Firestore are rejected by security rules. • All fields use the correct Firestore data types (String, Number, Boolean). • Documents are keyed by the Firebase Auth UID for all collections.	

US-08: Bottom Navigation Bar

Story ID: US-08 Story Title: Bottom Navigation Bar	**Priority:** Medium **Estimated Hours:** 3
User Story: As **a student,** I want **a bottom navigation bar with clearly labelled tabs** so that **I can easily switch between the main sections of the app.** **Description:** This story covers the MainActivity shell with a BottomNavigationView hosting five tabs: Home, Search, Chat, Requests, and Profile. Custom XML drawables are used for each tab icon. **Sub User Stories:** • As a student, I want five tabs: Home, Search, Chat, Requests, and Profile.	

• As a student, I want custom icons for each tab that clearly represent their function. • As a student, I want the currently selected tab to be visually highlighted. **Acceptance Criteria:** • All five tabs are visible and tappable in the bottom navigation bar. • Switching tabs loads the corresponding Fragment without restarting MainActivity. • The active tab icon and label are highlighted to indicate the current screen. • Custom drawable icons are displayed for all five tabs.	

3. Structured Specifications

Four structured specifications are provided below for selected user stories of Sprint 1.

3.1 User Registration (US-01)

Structured Specification — UC-01: User Registration	
Use Case ID	UC-01
Use Case Name	User Registration
Actor(s)	New Student (Primary)
Precondition	The user has not previously registered. The app is running and an internet connection is available.
Trigger	The user opens the app for the first time and navigates to the Sign Up tab on the Login screen.
Main Flow	• User opens the app and is presented with the Login/Sign Up screen. • User taps the Sign Up tab. • User enters a valid email address. • User enters a password of at least 6 characters. • User taps the Register button. • System creates a new account via Firebase Authentication. • System stores the UID in Firestore under the users collection. • System redirects the user to ProfileSetupActivity.
Alternative Flow	• 3a. Email format is invalid — system shows: 'Enter a valid email address.' • 4a. Password is fewer than 6 characters — system shows: 'Password must be at least 6 characters.' • 6a. Firebase returns 'email already in use' — system shows a toast with the error message. • 6b. No internet connection — system shows: 'Check your internet connection and try again.'
Postcondition	A new Firebase Auth account and a Firestore user document are created. The user is navigated to the profile setup screen.
Non-Functional	Registration must complete within 3 seconds on a standard 4G connection. Passwords must never be stored in plaintext.

3.2 Student Profile Creation (US-04)

Structured Specification — UC-04: Student Profile Creation	
Use Case ID	UC-04
Use Case Name	Student Profile Creation
Actor(s)	Newly Registered Student (Primary)
Precondition	The user is authenticated and has not yet completed profile setup.
Trigger	The user is redirected to ProfileSetupActivity immediately after successful registration.
Main Flow	• User is shown the Profile Setup screen. • User enters their full name. • User enters their age. • User optionally uploads a profile photo from the device gallery. • User writes a short bio (optional). • User selects their gender from a dropdown. • User taps Save Profile. • System writes the data to the users Firestore collection under the user's UID. • System navigates the user to PreferencesActivity.
Alternative Flow	• 2a. Name field is empty — system shows: 'Name is required.' • 3a. Age is not a valid number — system shows: 'Please enter a valid age.' • 4a. User skips photo upload — system uses a default avatar placeholder. • 8a. Firestore write fails — system shows: 'Failed to save profile. Please try again.'
Postcondition	User profile document is created in Firestore. The user is navigated to the lifestyle preferences setup screen.
Non-Functional	Profile photo uploads must not exceed 5 MB. The setup screen must be fully usable on screens 5 inches and larger.

3.3 Lifestyle Preference Setup (US-05)

Structured Specification — UC-05: Multi-Step Lifestyle Preference Setup	
Use Case ID	UC-05
Use Case Name	Multi-Step Lifestyle Preference Setup
Actor(s)	Authenticated Student (Primary)

Precondition	The user has completed profile creation and is authenticated.
Trigger	The user is navigated to PreferencesActivity after completing profile setup, or taps Edit Preferences from their profile.
Main Flow	• User is presented with the Preferences setup screen. • User selects a sleep schedule (Early Bird / Night Owl / Flexible). • User selects a cleanliness level from 1 (Very Relaxed) to 5 (Very Clean). • User enters their minimum monthly budget in PKR. • User enters their maximum monthly budget in PKR. • User enters their preferred city or area. • User sets their policy on smoking (Allowed / Not Allowed). • User sets their policy on pets (Allowed / Not Allowed). • User sets their policy on guests (Preferred / Not Preferred). • User taps Save Preferences. • System writes data to the preferences Firestore collection. • System navigates the user to the home screen.
Alternative Flow	• 4a. Budget min field is empty — system shows: 'Please enter your minimum budget.' • 5a. Budget max is less than budget min — system shows: 'Maximum budget must be greater than minimum.' • 11a. Firestore write fails — system shows a toast error and allows the user to retry.
Postcondition	A preferences document is saved to Firestore. The matching engine can use these values for compatibility scoring in Iteration 2.
Non-Functional	All preference fields must be saved atomically. Partial saves are not acceptable. The save operation must complete within 3 seconds.

3.4 Profile Edit & View (US-06)

Structured Specification — UC-06: Profile Edit and View	
Use Case ID	UC-06
Use Case Name	Profile Edit and View
Actor(s)	Authenticated Student (Primary)
Precondition	The user is logged in and has previously completed profile and preference setup.
Trigger	The user navigates to the Profile tab in the bottom navigation bar.

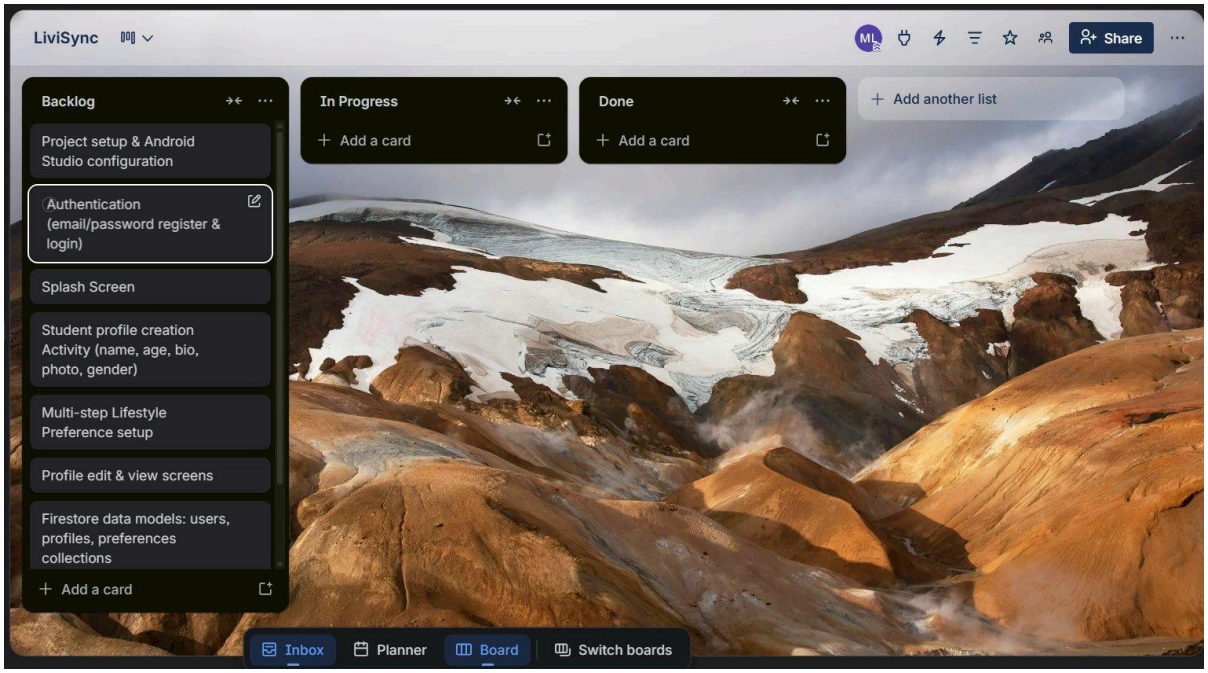
Main Flow	• User taps the Profile tab in the bottom navigation bar. • System fetches the user's data from Firestore and displays it on the Profile screen. • User views their name, bio, age, gender, sleep schedule, cleanliness, budget range, and policies. • User taps the Edit Profile button. • System opens EditProfileActivity pre-filled with existing data. • User modifies one or more fields. • User taps Save Changes. • System performs a Firestore batch write updating both the users and preferences collections. • System returns the user to the Profile view with the updated data displayed.
Alternative Flow	• 6a. User clears the name field — system shows: 'Name cannot be empty.' • 8a. Firestore batch write fails — system shows: 'Failed to save profile. Please try again.' • 2a. Firestore fetch fails — system shows an error message and a retry button.
Postcondition	Updated profile and preference data is reflected in Firestore and immediately visible on the Profile view screen.
Non-Functional	The profile view must load data within 2 seconds. Batch writes must be atomic — either all fields are updated or none are.

4. Scrum Board — Trello Snapshots

The sprint was tracked on Trello using three columns: Backlog, In Progress, and Done. Three snapshots were captured at the start, midpoint, and end of Sprint 1.

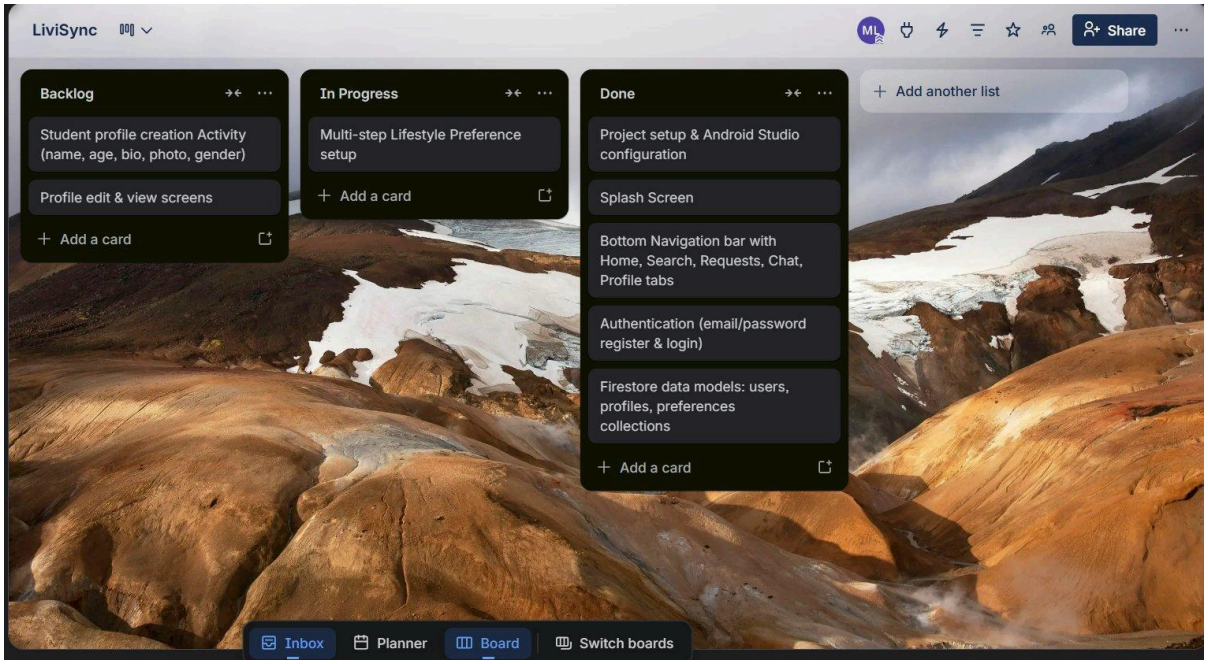
Snapshot 1 — Sprint Start (Day 1)

All seven tasks are loaded into the Backlog column. No tasks have been started. The board reflects the complete scope of Sprint 1 before any work begins.



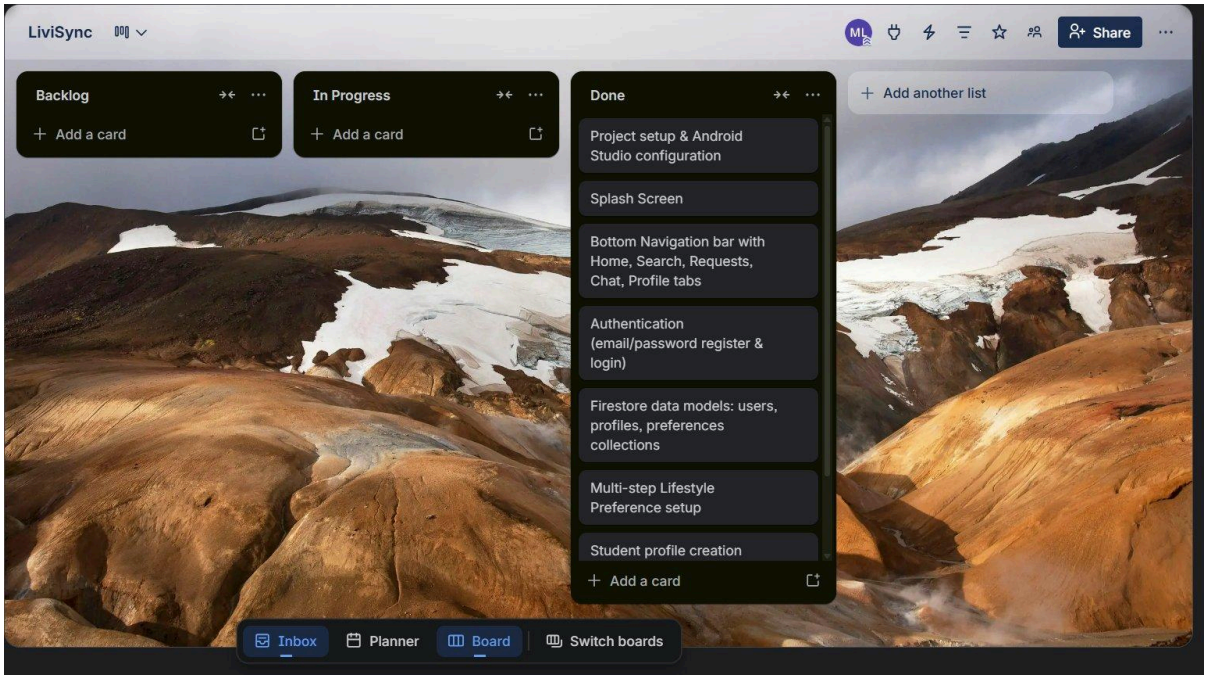
Snapshot 2 — Mid Sprint (Day 7)

Five tasks have been completed and moved to Done: Project Setup, Splash Screen, Bottom Navigation Bar, Authentication, and Firestore Data Models. Multi-step Lifestyle Preference Setup is In Progress. Student Profile Creation and Profile Edit & View remain in the Backlog.



Snapshot 3 — Sprint End (Day 14)

All planned tasks have been completed and moved to the Done column. Both the Backlog and In Progress columns are empty, confirming a successful sprint with zero leftover items carried forward to Iteration 2.



5. Sprint 1 Burndown Chart

The table below tracks the number of remaining tasks (planned vs. actual) over the 14-day Sprint 1 period. The sprint began with 7 tasks. The team front-loaded infrastructure tasks (project setup, authentication, Firestore models) in the first week, resulting in a slightly slower initial burn rate. Profile-related tasks were completed rapidly in the second week, bringing the actual line to zero by Day 13, one day ahead of the sprint deadline.

Day	Day 1	Day 3	Day 5	Day 7	Day 9	Day 11	Day 13	Day 14
Planned	7	6	5	4	3	2	1	0
Actual	7	7	6	2	2	1	0	0

Legend: Planned = ideal linear burndown from 7 to 0. Actual = real tasks remaining at each checkpoint.

Key observation: The actual burndown crossed below the planned line on Day 7 when five tasks were simultaneously moved to Done after the core infrastructure was completed. The remaining profile tasks were delivered ahead of schedule.

6. Non-Functional Requirements (NFR) Specification

The following non-functional requirements are specified using natural language for the LiviSync application. These requirements govern the quality attributes of the system and apply across all iterations.

1. Performance

The LiviSync application must respond quickly and smoothly under normal operating conditions. The splash screen must transition to the appropriate destination screen within 2 seconds on a mid-range Android device running on a standard 4G connection. Authentication operations including login and registration must complete within 3 seconds. Profile and preference data must be fetched from Firestore and rendered on screen within 2 seconds. All Firestore write operations such as saving a profile or preferences must complete within 3 seconds. The application must maintain a minimum frame rate of 30 FPS during all screen transitions and UI interactions.

2. Usability

The application must be intuitive and accessible to university students with no prior experience using roommate-matching platforms. A new user must be able to complete registration, create a profile, and configure their lifestyle preferences within 5 minutes without any external guidance. All input forms must provide immediate inline validation feedback when a field fails validation so that the user understands what correction is needed. The multi-step preference setup must use clear labels, descriptive dropdown options, and logical field ordering to guide the user without confusion. All error messages must be written in plain, non-technical language. Bottom navigation tab icons must be accompanied by text labels and must clearly represent their function.

3. Scalability

The Firestore data models and application architecture must be designed to support growth beyond the current user base without requiring structural redesign. The users and preferences collections must be able to handle at least 10,000 documents without degradation in read or write performance. The compatibility scoring engine, to be implemented in Iteration 2, must be designed to process comparisons across all registered users without requiring data restructuring. The codebase must follow a modular architecture — with separate Activities, Fragments, and utility classes — so that new features in Iterations 2 and 3 can be added without modifying existing components.

4. Reliability

The LiviSync application must operate consistently and recover gracefully from failures. The app must not crash more than once in 5 hours of continuous use under normal network conditions. All Firestore write operations for profile and preference data must be atomic — either all fields are saved successfully or none are, preventing partial or corrupted data states. In the event of a network failure during a save operation, the system must display a clear error message and provide the user with a retry option. Firebase Authentication session tokens must be managed automatically so that users are not unexpectedly logged out during active use. The application must handle Firestore permission errors gracefully by displaying a user-friendly message rather than crashing.

5. Security

The LiviSync application must protect user data and enforce appropriate access controls at all times. Firestore security rules must restrict read and write access to authenticated users only, ensuring that unauthenticated requests are rejected at the database level. User passwords must be managed exclusively through Firebase Authentication and must never be stored, logged, or transmitted in plaintext by the application. Profile photos uploaded to Firebase Storage must be accessible only to authenticated users. The application must not expose any Firebase configuration credentials in client-side code that is publicly accessible. All communication between the application and Firebase services must occur over HTTPS.

6. Compatibility

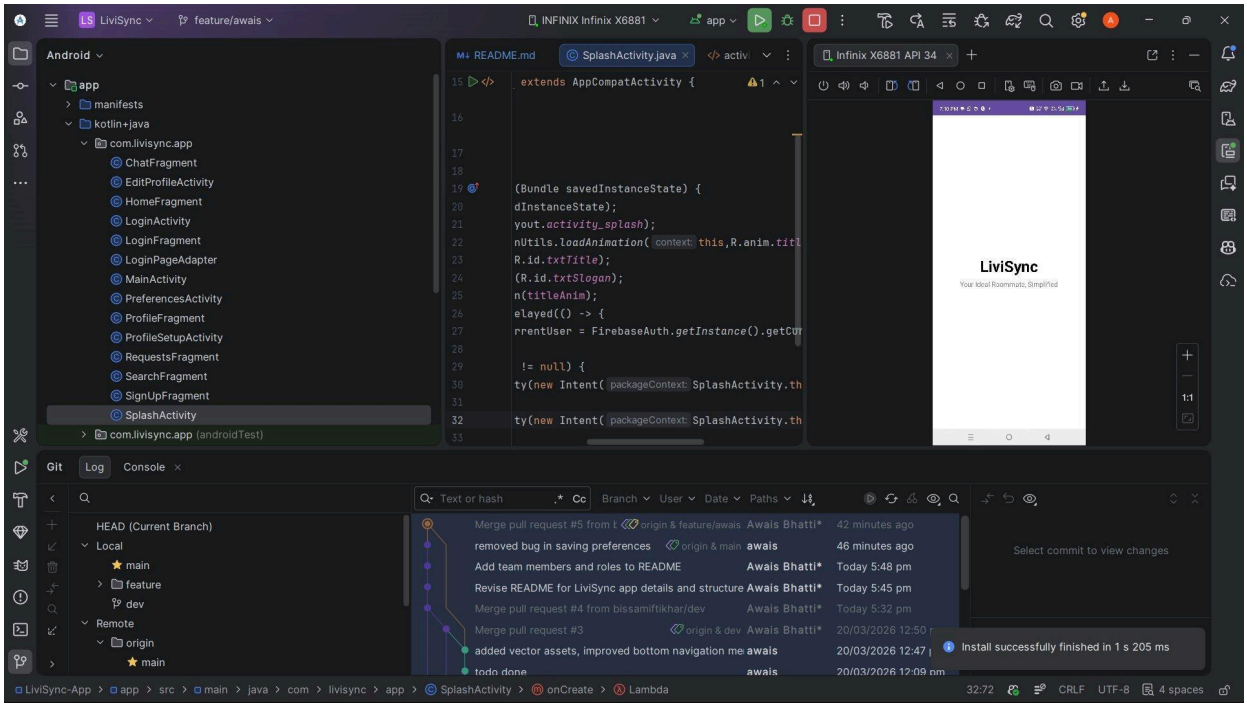
The LiviSync application must function correctly across a range of Android devices and software versions. The minimum supported Android version is Android 8.0 (API level 26), and the application must be fully functional on all versions up to Android 14 (API level 34). The UI layouts must be responsive and render correctly on screen sizes ranging from 5 inches to 7 inches and on both portrait and landscape orientations. The application must behave consistently across different Android manufacturers including Samsung, Xiaomi, and INFINIX devices, which vary in their system UI implementations. The application must not rely on any device-specific APIs that limit compatibility to a single manufacturer or Android variant.

7. Iteration 1 — Implementation Screenshots

The following screenshots demonstrate the completed Sprint 1 implementation running on an device via Android Studio. All core features of Module 1 are functional.

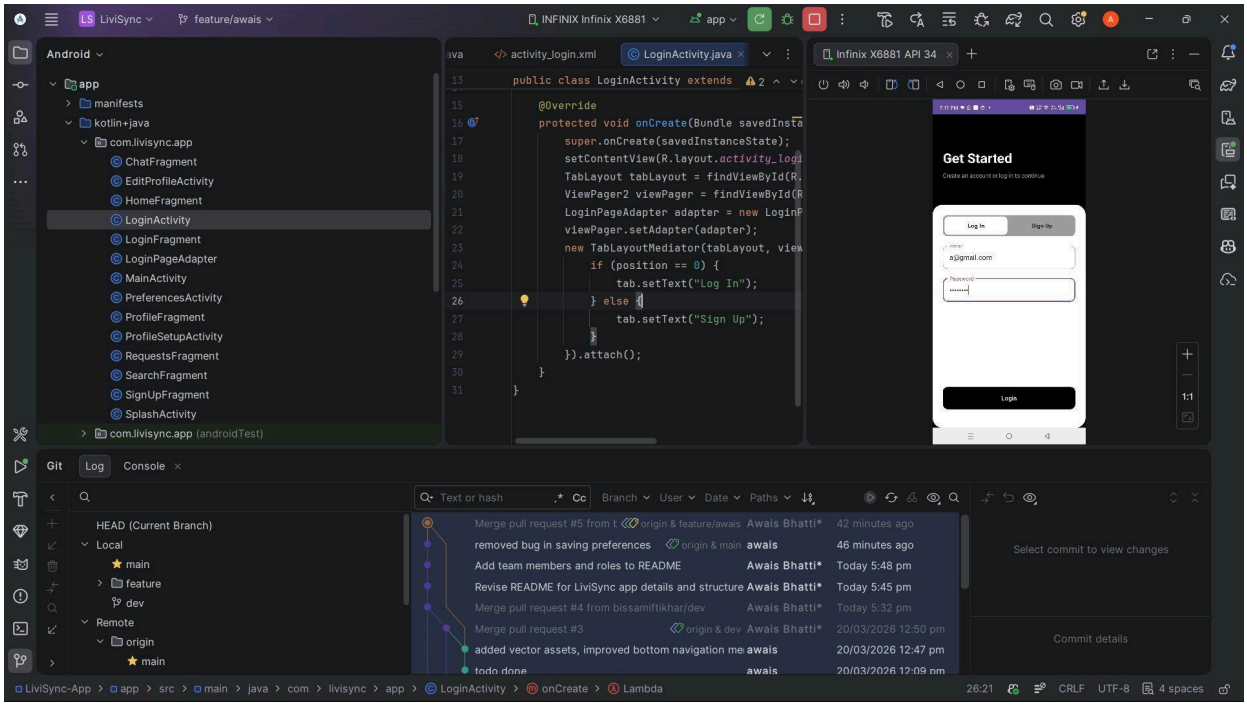
Screenshot 1 — Splash Screen

The animated splash screen displaying the LiviSync logo and slogan 'Your Ideal Roommate, Simplified.' The screen fades in on launch and routes the user based on their Firebase Auth session state.



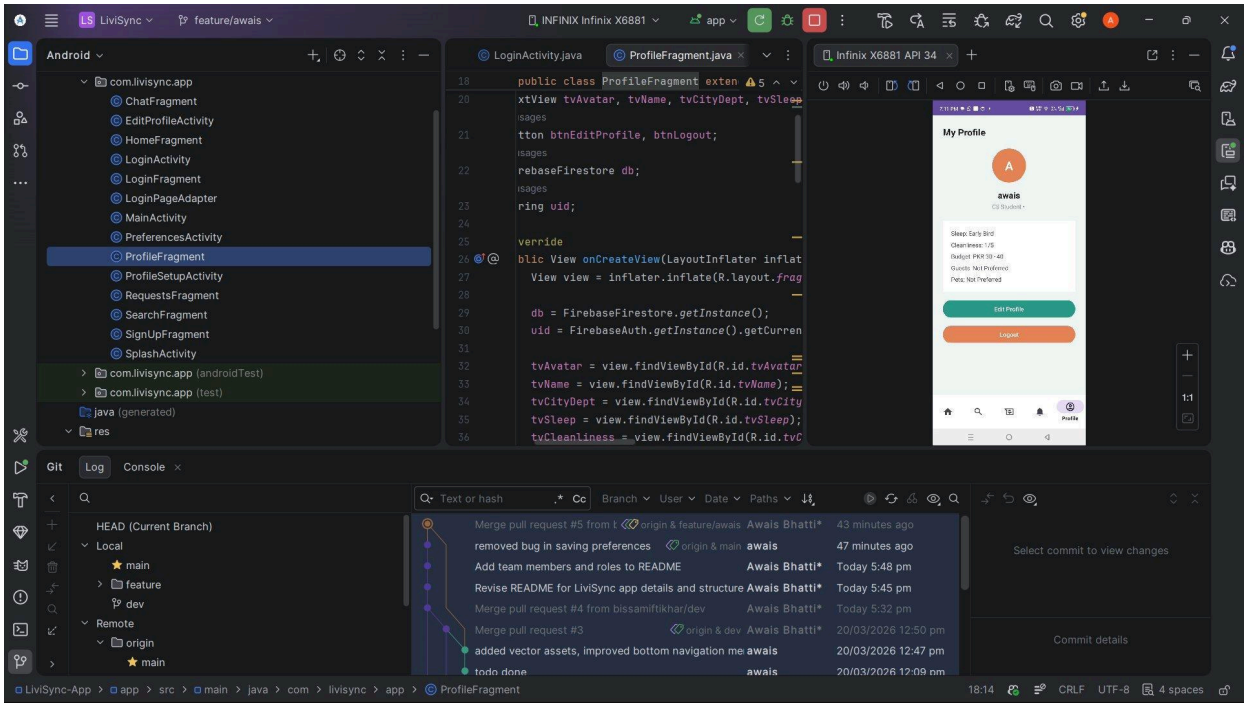
Screenshot 2 — Login / Sign Up Screen

The Login screen hosted inside LoginActivity using a ViewPager2 with two tabs: Log In and Sign Up. The screen shows the Get Started header, email and password fields, and the Login button. The Sign Up tab provides the registration form.



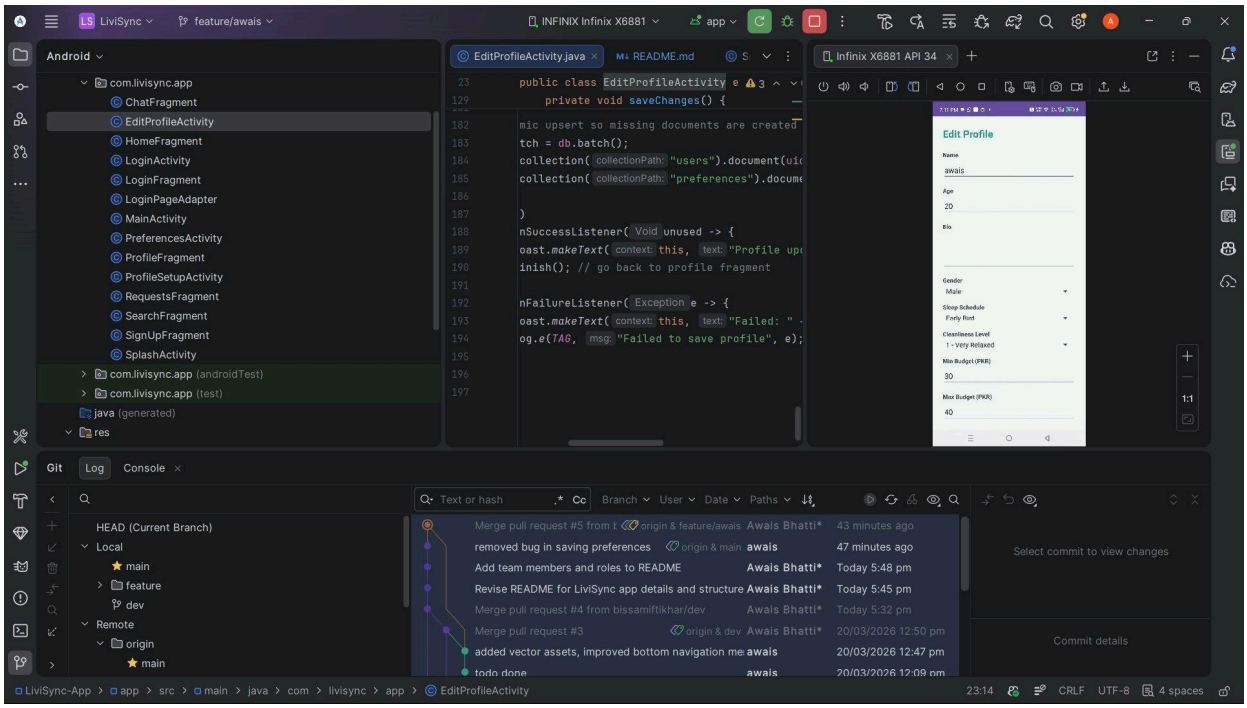
Screenshot 3 — Profile View Screen

The Profile tab showing the authenticated user's profile. The screen displays the user's avatar, name, city and department, and their lifestyle preferences including sleep schedule, cleanliness, budget range, guest preference, and pet policy. Edit Profile and Logout buttons are visible at the bottom. The bottom navigation bar is also active.



Screenshot 4 — Edit Profile Screen

The Edit Profile screen (EditProfileActivity) showing pre-filled fields including name, age, bio, gender dropdown, sleep schedule dropdown, cleanliness level dropdown, and budget range fields. All changes are batch-written to both the Firestore users and preferences collections on save.



8. Work Division

The following table outlines the Sprint 1 responsibilities assigned to each team member. Work was divided based on the roles defined in the project plan and coordinated through GitHub branching and Trello task management.

Team Member	Role	Sprint 1 Responsibilities
Awais Akhtar(23L-0792)	Project Lead & Backend Developer	• Firestore data model design (users and preferences collections) • SplashActivity implementation and session routing logic • ProfileFragment and EditProfileActivity Java implementation • PreferencesActivity Java logic and Firestore write operations • Trello board maintenance and sprint progress tracking • User manual and README documentation • Integration of all feature branches and final APK build for Sprint 1
Muaz Arshad(23L-0816)	Frontend Developer & Requirements Engineer	• User story writing, acceptance criteria definition, and feature scope • XML layout design for all activities and fragments (Login, Profile, Edit Profile, Preferences) • Color schemes, typography, iconography, and bottom navigation custom drawables • ViewPager2 implementation for Login/SignUp tabs (LoginPageAdapter) • Test case creation for authentication and profile flows
Bissam Iftikhar(23L-0913)	QA Tester & Android Developer	• Overall project coordination and GitHub branching strategy • Firebase project setup and Firestore security rules configuration • Backend logic for profile and preference save operations • Chat, Search, Requests, and Home fragment placeholder UI • Bug tracking and regression testing across all Sprint 1 features

		• Device compatibility testing on INFINIX X6881 (Android API 34)